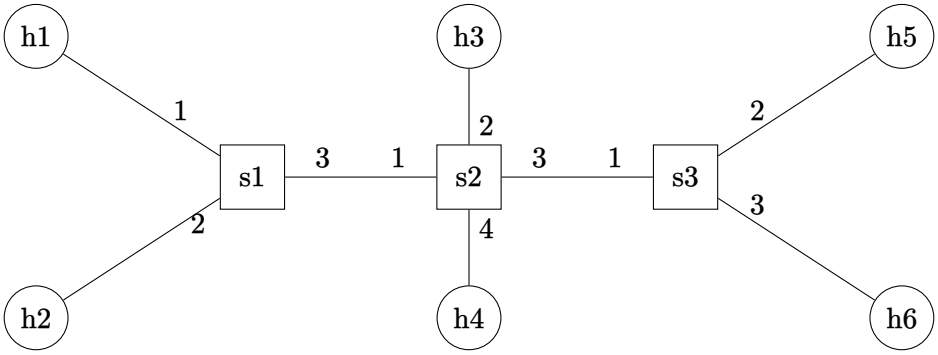
Software Defined Networking
22 gennaio 2024

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Figura 1.

L'host h1 porta 1234 invia un datagramma UDP verso h4 porta 5678.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** S1 invia un messaggio packet-in e riceve in risposta una flow-mod e una packet-out.

2. Scrivere con quale TTL il pacchetto arriva alla destinazione.

> **Soluzione:** 200

L'host h1 porta 1234 invia un altro datagramma UDP verso h4 porta 5678.

3. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** S1 usa la nuova regola e non invia messaggi al controllore

L'host h1 porta 1234 apre una connessione TCP verso h4 porta 5678.

4. Scrivere quali host collegati ad s1 ricevono il pacchetto e con quale TTL.

Soluzione: Nessun host. S1 usa la nuova regola e invia il pacchetto soltanto a s2.

5. L'host che riceve la richiesta di apertura, risponde. Scrivere quali host collegati ad s1 ricevono il pacchetto di risposta e con quale TTL.

Soluzione: h1 con TTL = 199. La nuova regola in s2 decrementa il TTL.

6. Scrivere la regola da aggiungere in s2 per fare in modo che le richieste http verso h3 siano inviate ad un web server in esecuzione su h5.

Soluzione:

Priority	Match	Action
20	eth_type = IPv4 ipv4_dst = IP3 ip_proto = TCP tcp_dst = 80	SetField(eth_dst=MAC5) SetField(ipv4_dst=IP5) Output(3)

7. Scrivere la regola da aggiungere in s2 per gestire la risposta.

Soluzione:

Priority	Match	Action
20	eth_type = IPv4 ipv4_src = IP5 ip_proto = TCP tcp_src = 80	SetField(eth_src=MAC3) SetField(ipv4_src=IP3) Output(FLOOD)

Switch s1

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4	Output(CONTROLLER)
15	eth_dst = MAC2	Output(2)
15	eth_dst = MAC1	Output(1)

Switch s2

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto = TCP	Output(CONTROLLER)
10	eth_type = IPv4 ip_proto = UDP	Output(FLOODING)

Switch s3

Priority	Match	Action
5	*	DecTTL Output(FLOOD)
15	eth_dst = MAC5	Output(2)
15	eth_dst = MAC6	Output(3)

Figura 1: Regole iniziali

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
   msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
   parser = datapath.ofproto_parser; in_port = msg.match['in_port']
   pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
   pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
   pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
   data = msg.data

   if (pkt_udp is not None) and (datapath.dpid == 1):
      match = parser.OFPMatch(
         eth_src=pkt_eth.src, eth_dst=pkt_eth.dst )
      actions = [
         parser.OFPDecTTL(), parser.OFPActionOutput(3) ]
      inst = [parser.OFPInstructionActions(
         ofproto.OFPIT_APPLY_ACTIONS, actions)]
      mod = parser.OFPFlowMod(datapath=datapath, priority=20,
         match=match, instructions=inst)
      datapath.send_msg(mod)

   if (pkt_tcp is not None) and (datapath.dpid == 2):
      match = parser.OFPMatch(
         eth_dst=pkt_eth.src, eth_src=pkt_eth.dst,
         eth_type=ETH_TYPE_IP, ip_proto=IPPROTO_TCP)
      actions = [
         parser.OFPDecTTL(),
         parser.OFPActionOutput(in_port) ]
      inst = [parser.OFPInstructionActions(
         ofproto.OFPIT_APPLY_ACTIONS, actions)]
      mod = parser.OFPFlowMod(datapath=datapath, priority=20,
         match=match, instructions=inst)
      datapath.send_msg(mod)

   actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
   out = parser.OFPPacketOut(datapath=datapath,
      in_port=in_port, actions=actions)
   datapath.send_msg(out)
```

Esercizio 2

Un sistema di blocco di URL fraudolente è costituito da un filtro di Bloom a conteggio seguito da una hash table. Tutte le URL fraudolente sono inserite sia nel filtro di Bloom che nella hash table. Quando si tenta di accedere a una URL, si verifica se essa è presente nel filtro di Bloom. In caso positivo, allora si verifica se è presente anche nella hash table. Se la URL è presente sia nel filtro di Blomm sia nella hash table allora la URL viene bloccata. Altrimenti la URL non è bloccata e si procede normalmente.

Il filtro di Bloom è costituito da un array di $m = 2^{14}$ bit e $k = 3$ funzioni hash. La hash table è costituita da un array di $\mu = 2^{10}$ elementi. In caso di collisione, le URL sono inserite in una lista. Si consideri la situazione dopo aver effettuato l'inserimento di 10 000 URL fraudolente.

8. Calcolare la probabilità di falso positivo a valle del filtro di Bloom.

Soluzione:

$$\left(1 - \exp\left(-\frac{kn}{m}\right)\right)^k = 59\%$$

9. Calcolare la probabilità di falso positivo a valle della tabella hash.

Soluzione: 0

10. Calcolare la somma dei valori dei contatori del filtro di Bloom.

Soluzione: $kn = 30000$

11. Calcolare il numero medio di accessi in memoria per rimuovere una URL dal filtro e dalla tabella.

Soluzione: k accessi per il filtro di Bloom.

Per la hash table, con in media $n/\mu = 9{,}7$ accessi.

Mettendo insieme le due cose abbiamo:

$$k + \frac{c}{\mu} = 3 + 9{,}7 = 12{,}7$$

12. Calcolare il numero ottimo di funzioni hash per il filtro di Bloom.

Soluzione:

$$k = \frac{m}{n} \log 2 \simeq 1$$

Esercizio 3

Una matrice di commutazione crossbar di uno switch 4×4 usa un algoritmo PIM per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto. Nel caso in cui si debba operare una scelta casuale, l'algoritmo sceglie sempre l'identificativo minore.

Ingresso 1: 3, 2, 3

Ingresso 2: 1, 2, 3

Ingresso 3: –, 4, 2

Ingresso 4: –, 4, 1

13. Quali pacchetti sono trasmessi durante il primo round? Indicare le coppie ingresso, uscita.

> **Soluzione:** Ingresso 2 uscita 1
> Ingresso 1 uscita 2
> Ingresso 3 uscita 4

14. Quali pacchetti sono trasmessi durante il secondo round?

> **Soluzione:** Ingresso 4 uscita 1
> Ingresso 2 uscita 2
> Ingresso 1 uscita 3

Si faccia riferimento all'algoritmo PIM con assegnazione casuale uniforme.

15. Qual è la proabilità che l'uscita 3 scelga l'ingresso 1 al primo round?

> **Soluzione:** L'uscita riceve una richiesta dall'ingresso 1 e una dall'ingresso 2. La probabilità che scelga l'ingresso 1 è $1/2$.